

Cursor AI 연습 #1~#4

VSCode C++ & Java 개발자 통합 교안 | Best Reviewer 양성과정

#1

TDD_TV 코드 리뷰

브랜치 전략 · 프롬프트 엔지니어링 · SOLID 리뷰 · PR 가이드

#2

SHealth BMI 분석

코드 스멜 탐지 · 리팩토링 · 단위 테스트 · 기능 개선

#3

UnitConverter PRD

PRD 기반 구현 · OCP/SRP 설계 · 외부 설정 · 출력 포맷

#4

Feedback Analyzer

레거시 이해 · 테스트 추가 · 버그 수정 · 추가 기능 개발

공통 실습 환경 설정 — VSCode C++ & Java

Cursor AI 공통 설정

.cursorrules (프로젝트 루트)

기술 스택

- Java 21 / C++17
- JUnit 5 + Mockito / Google Test
- Maven / CMake
- VSCode (Cursor AI)

코드 규칙

- 매직 넘버 → 상수로 추출
- 모든 public 메서드에 Javadoc / Doxygen
- 테스트 메서드명: should_[결과]_when_[조건]
- 한 함수 최대 20줄

Git 공통 브랜치 전략

연습별 브랜치 구조

연습 #1 (TDD_TV)

refactoring → prompting

연습 #2 (SHealth)

main → refactoring → tc → feature

연습 #3 (UnitConverter)

main → feature → tc → refactoring

연습 #4 (FeedbackAnalyzer)

main → tdd → refactoring → feature

공통: merge 하지 않고 PR + Review만 수행

추천 브랜치 전략

main ←— (릴리스 태그/머지 확인)

↑

staging

↑

A_01 ←— spec

←— red

←— green

←— refactoring

←— feature/new_feature

💡 모든 연습은 Conventional Commits 형식으로 커밋하세요: feat: / fix: / refactor: / test: / docs:

공통 실습 환경 설정 — VSCode C++ & Java



C++ 환경 (연습 #2 BMI 적용 가능)

VSCode C++ 확장 & CMake

필수 확장

C/C++ (ms-vscode.cpptools)
CMake Tools (ms-vscode.cmake-tools)
C++ TestMate (테스트 UI 연동)

CMakeLists.txt (Google Test 연동)

```
find_package(GTest REQUIRED)
target_link_libraries(test_target GTest::gtest_main)
```

빌드 & 테스트

```
cmake -B build && cmake --build build
cd build && ctest --output-on-failure
```



Java 환경 (모든 연습 공통)

VSCode 필수 확장 & 설정

1. 필수 Extension 설치

Extension Pack for Java (vscjava.vscode-java-pack)
→ Language Support, Debugger, Test Runner, Maven 포함

2. 빌드 도구

Maven for Java 또는 Gradle for Java

3. 테스트 실행

Test Runner for Java (JUnit 5 지원)
→ 사이드바  → Run All Tests

4. .vscode/settings.json

```
{
  "java.test.defaultConfig": "junit5",
  "editor.formatOnSave": true,
  "java.compile.nullAnalysis.mode": "auto"
}
```

연습 #1

TDD_TV 코드 리뷰 & 프롬프트

Cursor AI로 기존 프로젝트 전면 검토

1

연습 #1 — 브랜치 설정 & VSCode에서 프로젝트 열기

연습 #1

브랜치 생성 순서 (Git Bash)

bash - TDD_TV 브랜치 설정

1. 프로젝트 클론

```
git clone https://github.com/[org]/TDD_TV_xx.git
cd TDD_TV_xx
```

2. 모든 원격 브랜치 가져오기

```
git fetch --all
for branch in $(git branch -r | grep -v HEAD); do
  git checkout -b ${branch#origin/} ${branch}
done
```

3. refactoring 브랜치에서 새 브랜치 생성

```
git checkout refactoring
git checkout -b prompting
```

4. 초기 커밋 & Push

```
git add .
git commit -m "chore: Cursor AI 연습 브랜치 초기화"
git push --set-upstream origin prompting
```

5. VSCode에서 프로젝트 열기

```
code . # 또는 Cursor에서 File > Open Folder
```

Cursor AI에서 프로젝트 분석 시작

Chat (Ctrl+L) - 첫 번째 분석 프롬프트

@Codebase 이 Java 프로젝트의 전체 구조를 분석해줘:

- 주요 클래스 역할과 책임
- 클래스 간 의존성 관계
- State 패턴 구현 방식
- Test Double 사용 패턴

결과를 analysis.md 파일로 저장해줘.

C++17 병행 실습 포인트

C++ 동일 구조 재구현

```
// ITuner.h - 순수 가상 인터페이스
class ITuner {
public:
    virtual ~ITuner() = default;
    virtual std::string seekCH() = 0;
    virtual void setCH(std::string ch) = 0;
    virtual std::string getCurrentCH() = 0;
};
// TVChannelController.h - 동일 패턴 적용
```

 Cursor Chat에서 @Codebase를 사용하면 프로젝트 전체를 컨텍스트로 참조합니다. 첫 분석 후 analysis.md를 기준으로 리뷰를 진행하세요.

연습 #1 — 단계별 코드 리뷰 프롬프트 엔지니어링

연습 #1

1단계

컨텍스트 명확화

```
@TVChannelController.java @TVControllerTest.java
다음 관점으로 리뷰해줘:
[배경] TDD로 개발된 채널 컨트롤러 (Java 21, JUnit 5)
[리뷰 범위]
1. Unit Test 품질 및 커버리지
2. SOLID 원칙 준수 여부
3. 테스트 가능성(Testability)
[출력] 항목별 표 + Before/After 코드 예시
```

2단계

SOLID 원칙별 상세 리뷰

```
@Codebase SOLID 원칙 관점에서 평가해줘:
S - TVController가 단일 책임만 갖는가?
    채널관리/선호채널/입력처리가 분리됐는가?
O - 새 버튼 기능 추가 시 기존 코드 수정 없이 가능한가?
L - ITuner 구현체 교체 시 문제없는가?
I - 미사용 인터페이스 메서드 의존이 없는가?
D - 구체 클래스가 아닌 추상화에 의존하는가?
각 원칙별 1~10점 + 개선 코드 제시해줘.
```

연습 #1 — 단계별 코드 리뷰 프롬프트 엔지니어링

연습 #1

3단계

종합 리뷰 & 개선 우선순위

@Codebase QA 종합 리뷰를 Markdown 보고서로 작성해줘:

1. 요구사항 충족도 (README.md 기준 매핑)
2. AAA 패턴 준수도
3. 경계값 테스트 존재 여부
4. 개선 권고사항 (Critical/Major/Minor 분류)
5. 잘 구현된 부분 언급

출력: # 보고서제목 ## 섹션 형식 Markdown

마지막에 "이 결과를 review_report.md에 저장해줘"



프롬프트는 단계별로 나누어 실행하세요. 한 번에 모든 것을 요청하면 품질이 낮아집니다. 영역별 순차 실행 → 결과 문서화 → 개선점 도출.

연습 #1 — 10영역 체계적 리뷰 & QA PR 가이드

연습 #1

📋 10영역 체계적 리뷰 순서 (Cursor Chat)

- | | | |
|---|------------|--------------------------------|
| ① | 아키텍처 분석 | @Codebase 전체 구조·의존성 분석 |
| ② | 요구사항 추적성 | README ↔ 구현 코드 일치 검증 |
| ③ | Clean Code | SRP·매직넘버·중복·메서드 길이 |
| ④ | SOLID 원칙 | 5원칙 각각 코드 평가·점수 |
| ⑤ | Test Smell | Obscure/Eager/Mystery Guest 탐지 |
| ⑥ | 기능 시나리오 | 숫자버튼·선호채널·채널검색 커버 |
| ⑦ | 예외/경계조건 | CORRECT 원칙 적용 경계값 테스트 |
| ⑧ | 보안·동시성 | Thread Safety·입력 검증·예외 노출 |
| ⑨ | 유지보수성 | 확장성·의존성·순환 의존성 검사 |
| ⑩ | 문서화 | Javadoc·네이밍·README 일관성 |



리뷰어 등록 필수! PR 생성 시 반드시 팀원을 Reviewer로 지정하세요. Cursor Chat으로 리뷰 코멘트 초안을 작성한 뒤 직접 게시하세요.

🔍 QA 관점 PR 리뷰 체크리스트

기본 정보

- ☐ PR 제목이 변경사항을 설명하는가?
- ☐ 관련 이슈 번호가 연결됐는가?
- ☐ 400라인 이하의 적절한 크기인가?

테스트 품질 (FIRST)

- ☐ Fast: 빠르게 실행되는가?
- ☐ Independent: 서로 독립적인가?
- ☐ Self-validating: 자동 성공/실패 판단?

SOLID 준수

- ☐ SRP: 단일 책임 원칙 준수?
- ☐ OCP: 수정 없이 확장 가능?
- ☐ DIP: 추상화에 의존?

코드 스멜

- ☐ 중복 코드 제거됐는가?
- ☐ 의미있는 변수명/함수명?

연습 #2

SHealth BMI 분석 & 리팩토링

코드 스멜 탐지 → 리팩토링 → 단위 테스트 → 기능 개선

2

연습 #2 — SHealth BMI 프로젝트 개요 & 6시간 Activity

연습 #2

SHealth BMI 요구사항 개요

 나이대(20·30·40대)별 저체중/정상/과체중/비만 통계 계산

 누락 체중(weight=0): 동일 연령대 평균으로 보정

 $BMI = \text{체중(kg)} / \text{키(m)}^2$

 BMI 기준: ≤ 18.5 저체중 | $18.5 < x < 23$ 정상 | $23 \leq x < 25$ 과체중 | ≥ 25 비만

 입력: shealth.dat (id,age,weight,height CSV 형식)

- 삼성 헬스에서는 수집된 데이터를 활용하여 사용자들의 나이대(ex. 20대, 30대, 40대 등)별 저체중/정상체중/과체중/비만의 통계를 계산하고자 합니다.
- 수집 중 누락된 체중값이 있으며 같은 나이대(ex. 20대, 30대, 40대 등)의 평균을 적용합니다. 체중 0이 누락된 경우입니다.
- 수집 데이터는 ID, 나이, 몸무게(kg), 키(cm)이며, BMI는 체중(kg)/키(m)제곱으로 계산합니다.
- BMI 기준으로 18.5이하 저체중, 18.5초과 23미만 정상체중, 23이상 25미만 과체중, 25이상 비만으로 판단합니다.
- 제공된 코드에는 다양한 코드 품질 문제가 있습니다.

 C++ 수강생은 STL(vector/map/algorithm) 적극 활용. Java 수강생은 Stream API·Optional을 활용해 리팩토링하세요.

연습 #2 — SHealth BMI 프로젝트 개요 & 6시간 Activity

연습 #2

C++ & Java 공통 입력 형식

shealth.dat

```
id,age,weight,height
93705,66,79.5,158.3
93708,66,53.5,150.2
93709,75,0,151.1 ← weight=0: 보정 필요
93712,25,68.0,0 ← height=0: 보정 필요
```

C++ 파싱 예시

```
// C++17 구조체 정의
struct HealthRecord {
    int id, age;
    double weight, height;
};
// std::ifstream으로 CSV 파싱
// 0값 → 동일 나이대 평균으로 보정
```

6시간 Activity 구성

1h

코드 스멜 탐지

구조 이해 + Bad Smell 목록화

1h

1차 리팩토링

네이밍·매직넘버·함수 추출·중복 제거

1h

단위 테스트 작성

BMI·보정·분류·예외 TC 작성

2h

기능 개선

SRP 리팩토링 + 4가지 신규 기능

1h

회고 & 발표

Before/After 비교·AI 활용 경험 공유

연습 #2 — Cursor AI 단계별 프롬프트 가이드

연습 #2

① 코드 스멜 탐지

⚙️ C++ - 동일 패턴 적용

```
// C++ 동일 프롬프트 적용
// constexpr로 상수 추출 예시:
constexpr double BMI_UNDERWEIGHT = 18.5;
constexpr double BMI_NORMAL_MAX  = 23.0;
constexpr double BMI_OVERWEIGHT  = 25.0;
```

🐼 Java - Cursor Chat 프롬프트

@Codebase 이 BMI 프로젝트의 코드 스멜을 찾아줘:

- Magic Number (18.5, 23, 25, 0 등 하드코딩)
- Long Method (20줄 이상 메서드)
- Duplicated Code (중복 로직)
- God Class (너무 많은 책임을 가진 클래스)

심각도 순으로 표 형식으로 정리해줘.

② TDD 테스트 먼저 작성

⚙️ C++ - 동일 패턴 적용

```
// Google Test 동일 패턴
TEST(BMICalculatorTest,
     ShouldReturnUnderweight_WhenBMIBelow18_5) {
    BMICalculator calc;
    EXPECT_EQ("저체중", calc.classify(18.0));
}
// 빌드: cmake --build build && ctest
```

🐼 Java - Cursor Chat 프롬프트

BMI 계산기에 대해 TDD를 시작할게.
실패하는 JUnit 5 테스트를 먼저 작성해줘:

1. BMI 계산 로직 (정상 입력)
2. 평균 보정 (weight=0, height=0)
3. BMI 카테고리 분류
4. 연령대별 통계 계산

각 테스트는 should_[결과]_when_[조건] 네이밍.

연습 #2 — Cursor AI 단계별 프롬프트 가이드

연습 #2

③ 리팩토링 & 기능 개선

⚙️ C++ — 동일 패턴 적용

```
// C++: 함수 분리 예시
double calculateBMI(double weight, double heightCm);
std::string classifyBMI(double bmi);
double correctWeight(int age,
    const std::vector<Record>& records);
// 각 함수에 Google Test 작성
```

🍵 Java — Cursor Chat 프롬프트

@BMICalculator.java 현재 코드를 클린코드 원칙으로 리팩토링해줘:

- Magic Number → 상수 추출
- 한 함수 하나의 책임 (SRP)
- 테스트하기 쉽도록 분리

추가 구현:

- 특정 연령대 BMI 분포 비율 계산
- BMI 정상 범위 사용자 목록 조회
- 전체 대비 각 카테고리 비율 계산

📝 각 단계 완료 후 *.md 파일 작성 필수: code_smell.md / refactoring.md / unittest.md / feature.md / retrospective.md

연습 #3

UnitConverter

PRD 기반 신규 구현

PRD → OCP/SRP 설계 → 외부 설정 → 다중 출력 포맷

3

강의

아름다운 소프트웨어를 위한 해석학적 접근

Concept-to-Code Traceability · 의사결정 파이프라인 · 문제인식 + 온톨로지 + 프롬프팅 체인

3.1 왜 설계 의도는 코드에서 사라지는가? — Semantic Erosion(의미침식)

2:3교시

"당신의 팀은 완벽한 요구사항 문서를 작성했다. 6개월 후 코드를 보니 설계 의도는 온데간데없고 스파게티만 남아 있다."



문서-코드 단절

개념 분석 산출물이
코드와 직접 연결 안 됨



테스트 수준 혼재

기술 단위 테스트만 작성
비즈니스 의미 검증 없음



UI의 도메인 잠식

UI 요구사항이 도메인
모델을 오염



아키텍처 우선 개발

아키텍처가 개념보다
먼저 결정됨

결과: "Semantic Erosion" — 시스템은 시간이 지날수록 의미적 일관성(Semantic Coherence)을 잃는다

도메인 로직 → UI 이동 · 컨트롤러 비대화 · 테스트가 구현 세부사항만 기술 · 원래 개념 모델 사라짐

3.2 문제인식(Problem Recognition) + 의사결정 파이프라인

2.3교시

문제인식이란: "언제 시스템이 실패했다고 말해야 하는가?"를 정의 — "무엇을 만들까"가 아닌 실패 조건부터

의사결정 파이프라인 전체 흐름

문제인식 → 의미화 → **온톨로지** → 디지털화 → 논리화 → 정규화 → 검증화 → 추상화 → 최적화 → 실행·결정

✗ 해결책 중심 (나쁜 문제인식)

- 책을 빌리는 기능이 필요하다
- 로그인 화면을 만들자
- 검색 기능을 추가하자

✓ 실패 조건 중심 (올바른 문제인식)

- 연체·한도 초과 시 → 대출 거부
- 이메일 오류·비밀번호 오류 각각 처리
- 검색어 0자·특수문자·결과 없음 정의

3.3 온톨로지(Ontology) — ECB 분석 모델로 세계를 구조화

온톨로지: "우리가 세계를 어떤 개념과 관계로 나누어 이해할 것인가"를 정의하는 명시적 모델 — 팔란티어의 성공 비결
예: '직원(객체)' 이 'OO 코리아(조직)'에서 근무한다

주요 온톨로지 예시

- 지식 그래프 및 의미망: '김철수(35세, 남, 테슬라 코리아 소속)'와 같이 객체 간의 복합적 관계를 그래프 형태로 표현하여 AI가 맥락을 이해하게 함.
- 분야별 온톨로지: 의학(질병, 증상, 약물 간 관계), 전자상거래(상품, 카테고리, 리뷰 데이터 간 관계), 에너지 산업(유정, 엔지니어, 상태 정보 간 공통 관점).
- 구조적 정의: "A는 B의 일종이다(is-a)", "A는 B를 가지고 있다(has-a)"와 같은 명세서.

<<Entity>>

Entity (엔티티)

시스템이 장기적으로 관리하는 도메인 정보

예: Member · Book · Loan · Order

💡 판단하지 않는다. 상태를 유지한다.

<<Control>>

Control (제어)

유스케이스를 수행하는 흐름 제어·업무 조정

예: LoanPolicy · LoginControl · OrderService

💡 판단한다. 조율한다. 상태는 최소화.

<<Boundary>>

Boundary (경계)

시스템과 외부 액터 사이의 접점

예: LoanUI · LoginAPI · ExternalPayment

💡 비즈니스 규칙을 포함하지 않는다.

3.4 Concept-to-Code Traceability — 정의와 4계층 모델

C2C Traceability 정의

문제 영역에서 식별된 개념(Concept)과 코드(Code) 사이의 의미적·구조적·행위적 연결이 명시적으로 유지·검증·진화 가능한 상태 — 의미 보존(Semantic Preservation) 목표



핵심 성질: 완전성(Completeness) · 일관성(Consistency) · 검증 가능성(Verifiability) · 진화 가능성(Evolvability)

3.5 프롬프팅 체인 — C2C Traceability의 현대적 구현

2.3교시

핵심 통찰: 프롬프트 엔지니어링 = C2C Traceability를 구현하는 현대적 방법론

프롬프트 = 명시적 의도 기록(Traceability Documentation) + 맥락 보존(Context Preservation) + 재생성 가능성(Reproducibility)

Layer 1

ECB 개념 분석

ECB 분석: 도서관 시스템
ENTITIES: Member, Book, Loan
CONTROLS: LoanPolicy, LoanService
BOUNDARIES: LoanUI, LoanAPI
각 개념의 ECB 스테레오타입과 책임을 정의하라



Layer 2

책임 시나리오

LoanPolicy(Control)의 책임:
1. 대출 가능 여부 판단
2. 대출 기간 계산
Trace: LoanPolicy는 Concept Layer의 Control



Layer 3

테스트 명세

LoanPolicy.can_borrow()
테스트:
- 정회원 + 0권 → True
- 정회원 + 3권 → False
Trace: Responsibility Layer의 "대출 가능 여부 판단"



Layer 4

코드 구현

위 테스트를 통과하는 LoanPolicy 구현
ECB: <<control>>
Trace:
Test→Responsibility
→Concept

각 프롬프트가 상위 계층을 명시적으로 참조 → Chain-of-Prompts = Traceability Chain → Traceability 자동 확보

3.6 ECB(Entity-Control-Boundary) 분석 패턴

개체 역할 기반 설계

E

Entity 엔티티

시스템이 장기적으로 보관해야 하는 데이터 정보

특징:

- 데이터 보유 (속성)
- 비즈니스 규칙 포함
- DB 테이블과 매핑

예시:

User, Product, Order
BookInfo, Inventory

C

Control 제어

유스케이스 흐름을 조정하는 비즈니스 로직

특징:

- Entity와 Boundary 연결
- 비즈니스 규칙 실행
- 트랜잭션 관리

예시:

AuthService, OrderProcessor
BookSearchController

B

Boundary 경계

시스템과 외부 액터(사람·시스템) 사이의 인터페이스

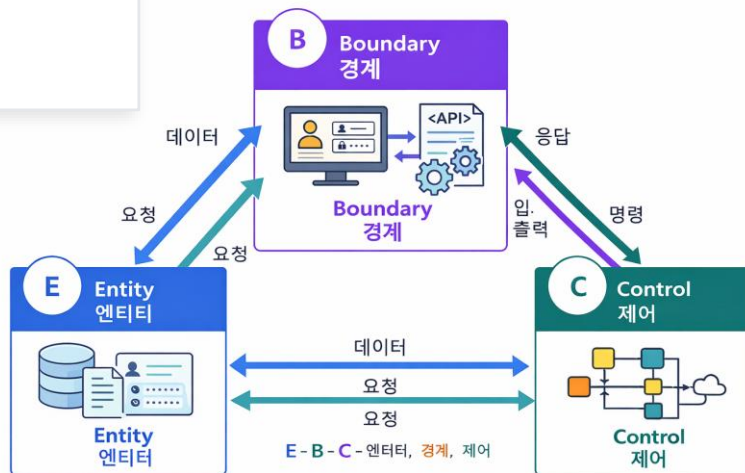
특징:

- UI 화면, API 엔드포인트
- 입출력 검증
- 외부와의 통신

예시:

LoginForm, ApiController
DashboardView, WebSocket

ECB 모델



ECB에서 Control이 비즈니스 규칙을 실행하고 Boundary가 외부와 통신한다면 — RBAC는 "누가 어떤 Control을 실행할 수 있는가"를 결정하는 접근 제어 레이어입니다

🔗 ECB 패턴에 RBAC를 더하면

- E Entity**
User·Role·Permission·Resource
→ RBAC 핵심 데이터 보관
- B Boundary**
로그인 폼·API 엔드포인트
→ Role 정보를 받아 Control로 전달
- C Control**
AuthService·PermissionChecker
→ Role-Permission 매핑·접근 허용/거부

🏠 RBAC 4대 구성 요소

- User** 시스템 사용자 — 개발자·운영자·관리자
- Role** 권한 묶음 단위 — admin·developer·viewer
- Permission** 자원에 대한 행동 — read·write·delete
- Resource** 접근 대상 — DB·API·파일·서비스

📌 용어 구분: 큰 틀 vs 세부 표현

- 큰 틀** → 역할 기반 접근 제어(RBAC)
- 메뉴·UI** → 권한(역할)에 따른 메뉴 가시성 제어
- 내비게이션** → RBAC에 따른 내비게이션 제어

3.8 RBAC 설계 패턴 — 권한에 따른 메뉴 가시성 제어

2:3교시

"권한에 따른 메뉴 가시성 제어" = Boundary(UI)가 Control(AuthService)에 Role을 질의해 메뉴를 보여줄지 숨길지 결정 — UX Contract의 '보인다/안 보인다' 언어와 직결

Role-Permission 매트릭스 (내비게이션 메뉴 기준)

역할	대시보드	사용자관리	설정	삭제
admin	✓	✓	✓	✓
developer	✓	✓	✗	✗
viewer	✓	✗	✗	✗
guest	⚠	✗	✗	✗

✓ UX Contract 연결 — "보인다 / 안 보인다" 규칙

Role=admin:	사용자관리 메뉴 보인다 · 삭제 버튼 활성화
Role=developer:	사용자관리 메뉴 보인다 · 삭제 버튼 비활성화
Role=viewer:	대시보드만 보인다 · 모든 관리 메뉴 숨김

Python — 메뉴 가시성 제어 구현

```
from enum import Enum

class Role(Enum):
    ADMIN = 'admin'
    DEVELOPER = 'developer'
    VIEWER = 'viewer'

# 역할별 메뉴 가시성 정의
MENU_VISIBILITY = {
    Role.ADMIN: {
        'dashboard': True,
        'user_mgmt': True,
        'settings': True,
        'delete_btn': True,
    },
    Role.DEVELOPER: {
        'dashboard': True,
        'user_mgmt': True,
        'settings': False,
        'delete_btn': False,
    },
    Role.VIEWER: {
        'dashboard': True,
```

R

RBAC (Role-Based Access Control) — 역할 기반 접근 제어

Superadmin, Admin, Manager, User, Viewer 등 역할에 따라 기능 접근을 제어

예: Admin만 사용자 관리 가능, Viewer는 읽기만 가능

권한(permission) 기반 작업 시 유지보수 수월

T

TBAC (Team-Based Access Control) — 팀 기반 접근 제어

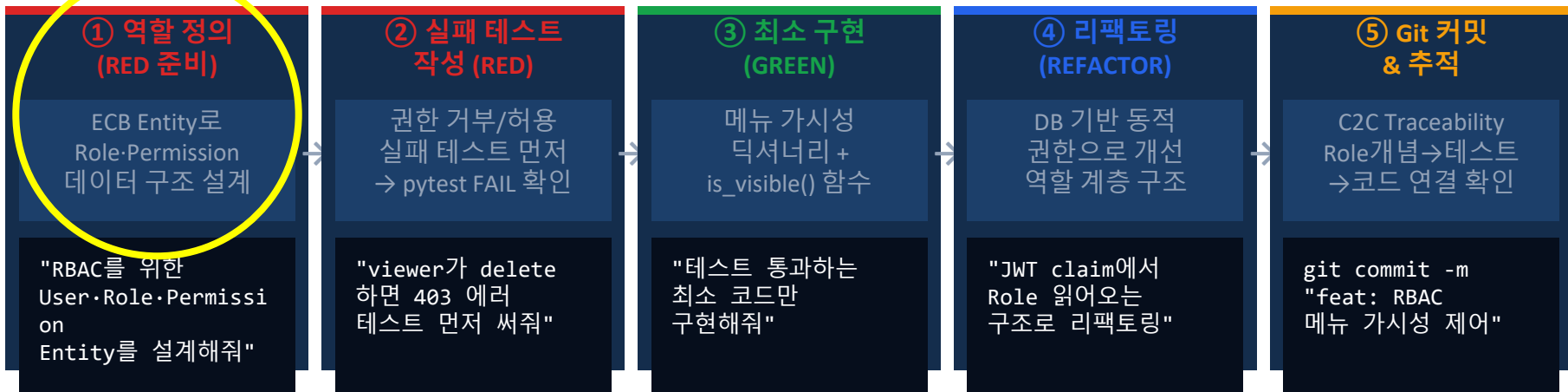
사용자가 소속된 팀에 따라 데이터 접근 범위를 제어

예: R&D 팀 소속 사용자는 R&D 팀 게시판/문서만 접근 가능

A

ABAC (Attribute-Based Access Control, 속성 기반 접근 제어)라고도 합니다. 팀 소속이라는 "속성"을 기준으로 접근 제어

RBAC를 TDD로 구현할 때 Cursor AI를 활용하는 전체 흐름 — C2C Traceability: 역할 개념 → 실패 테스트 → 최소 구현 → 리팩토링까지 의미 소실 없이 추적



● RED 단계 — 실패 테스트 예시

```
def test_viewer_cannot_see_user_mgmt():    # 메뉴 가시성
    assert not is_visible(Role.VIEWER, 'user_mgmt')

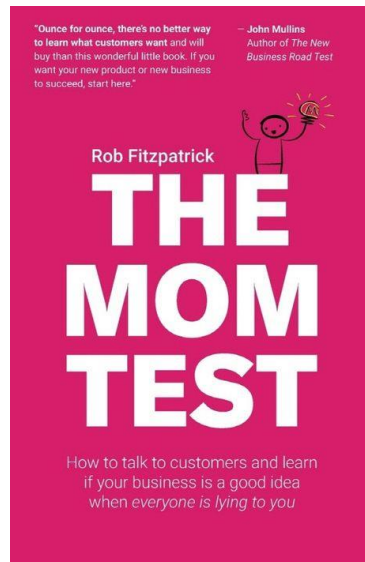
def test_admin_can_see_all_menus():        # admin 전체 접근
    assert all(is_visible(Role.ADMIN, m) for m in
MENU_VISIBILITY[Role.ADMIN])
```

📌 용어 정리

- ▶ 역할 기반 접근 제어(RBAC)
- ▶ 권한(역할)에 따른 메뉴 가시성 제어
- ▶ RBAC에 따른 내비게이션 제어

The Mom Test는 스타트업/제품 아이디어 검증에서 쓰이는 인터뷰 기법으로, 상대가 '좋은 사람이라서' 거짓 긍정(칭찬, 공감)을 해도 속지 않도록 질문하는 방법.

Mom Test = "사람의 말이 아니라 행동을 믿는 인터뷰 방법"



The Mom Test & 문제 인식 훈련

잘못된 질문의 예:

❌ "제가 만든 설비 모니터링 앱 어때요?"
→ 칭찬만 들을 것

❌ "이런 기능이 있으면 사용하시겠어요?"
→ 가짜 약속만 받을 것

❌ "이 앱에 얼마 지불하시겠어요?"
→ 현실성 없는 답변만 받을 것

The Mom Test & 문제 인식 훈련

올바른 질문의 예:

✓ "최근에 설비 점검하면서 가장 답답했던 순간이 언제였나요?"

→ 진짜 문제 발견

✓ "지금은 그 상황을 어떻게 해결하고 계세요?"

→ 현재 우회 방법 파악

✓ "그 방법의 어떤 점이 불편하세요?"

→ 개선 포인트 확인

The Mom Test & 문제 인식 훈련

핵심 원칙 3가지:

당신의 아이디어에 대해 말하지 마라
미래나 가정에 대해 묻지 마라
과거의 구체적인 행동에 대해 물어라

잘못된 데이터 피하기

피해야 할 3가지:

칭찬 (Compliments)

- 사용자: "오, 이거 정말 좋은 아이디어네요!"
- → 의미 없음. 행동으로 이어지지 않음
- 대신 물어볼 것:
- "비슷한 문제를 겪으신 적 있으세요? 그때 어떻게 하셨나요?"

가상의 약속 (Hypotheticals)

- 사용자: "그런 기능 있으면 꼭 쓸 것 같아요!"
- → 실제로는 안 씀
- 대신 물어볼 것:
- "현재 유사한 도구를 사용해보신 적 있나요? 얼마나 자주 쓰시나요?"

잘못된 데이터 피하기

일반론 (Generics)

- 사용자: "보통 사람들은 이런 거 좋아할 거예요"
- → 추측일 뿐

- 대신 물어볼 것:
- "선생님께서서는 직접 어떻게 하시나요?"

올바른 질문하기

좋은 질문의 특징:

과거의 구체적 사건

- "지난주에 설비 점검하실 때 어떤 일이 있었나요?"
- "그때 정확히 어떤 절차를 거치셨나요?"
- "얼마나 시간이 걸렸나요?"

돈/시간 투자 확인

- "현재 사용 중인 도구에 비용을 지불하고 계신가요?"
- "그 도구를 배우는 데 얼마나 시간을 쓰셨나요?"
- "팀원들도 다 쓰고 있나요?"

올바른 질문하기

해결 시도 이력

- "이 문제를 해결하려고 시도해본 적 있으세요?"
- "어떤 도구들을 써보셨나요?"
- "왜 계속 쓰지 않으셨나요?"

Dual-Track UI + Logic TDD

개발 방법론과 요구사항

왜 우리는 항상 늦게 실패하는가? · Problem & Scenarios · Toy Project

5.1 왜 우리는 항상 늦게 실패하는가? — 요구사항의 구조적 함정

5교시

"실패 조건이 정의되지 않으면, 성공은 합의될 수 없다"

기획서에는 기능이 있지만 실패 조건이 없다. 테스트는 항상 구현 후에 추가된다. 이것이 "늦은 실패"의 본질이다.

원인 1

기획은 있는데 설계는 없다

요구사항 문서 = 기능 목록
"실패 조건"이 없어 개발자마다 해석이
다름
→ 구현 후 "이게 아닌데" 반복

원인 2

테스트는 항상 늦게 추가된다

"구현 먼저, 테스트는 나중에"
→ 테스트가 구현의 하인이 됨
→ 비즈니스 의미를 잃음

원인 3

UI/Logic 분리의 오해

"UI 따로, 로직 따로" 개발
→ UI가 비즈니스 규칙 포함
→ 어느 것도 독립 테스트 불가

전통적 방식: 기획서→개발자 해석→구현→문제 발견 VS Dual-Track TDD: 실패 조건 합의→RED 테스트→구현

5.2 PRD Is the Code — Problem Recognition & Scenario (L0~L3)

5교시

"PRD는 문서가 아니라 테스트의 원천(Source of Truth)이다" — Problem + Scenario(L0~L3) + UX Contract + Logic Rule

PRD의 핵심 = 기능 목록이 아니라 실패 조건 목록

✗ 해결책 중심 PRD

- 로그인 기능을 만들자
- 책 검색 기능 추가
- 대출 한도 관리

✓ 실패 조건 중심 PRD

- 이메일 형식 오류 → 즉시 에러 표시
- 대출 3권 초과 → 거부 + 이유
- 연체 중인 회원 → 모든 대출 차단

② Scenario Level 0~3 — 실패를 시간축에 배열하다

L0

기능 개요

"회원이 책을 빌린다"

L1

Happy Path

정회원 + 가용 책 → 대출 성공

L2

Edge Case

대출 2권 → 성공 / 3권 → 거부

L3

Failure Path

연체 중 시스템 오류 권한 없음

5.2 PRD Is the Code — Problem Recognition & Scenario (L0~L3)

5교시

코딩 전 AI가 정확하게 실행할 수 있도록 명세서를 먼저 작성

PRD가 필요한 이유

- ▶ 방향성 없는 코딩 방지
- ▶ 명확한 기능 범위 설정
- ▶ 재작업 횟수 70% 감소
- ▶ 단계별 구현 계획 수립
- ▶ 팀 협업 공통 기반 제공

PRD 구성 요소

- ① 프로젝트 개요 (목적·배경)
- ② 타겟 사용자 정의
- ③ 핵심 기능 (Must/Should)
- ④ 기술 스택·제약 조건
- ⑤ 성공 지표 (KPI)

프롬프트

포트폴리오 웹페이지를 만들려고 해.
주요 기능은 채용 담당자가 30초 안에 내
역량을 파악할 수 있는 임팩트 있는 구성
이야.
프리랜서 전환을 준비하는 마케터를 위
한 것이고, 경쟁력 있는 지원자로 보이기
위해 필요해.
기술적인 방향과 함께 구체적인 PRD를
작성해 줘.

📌 실습 순서: PRD 작성 → 단계별 분해 → 단계별 실행 → 검토 요청 → 개선 요청 (5단계 패턴)

연습 #3 — UnitConverter 요구사항 & OCP/SRP 설계

연습 #3

실습 프로젝트(UnitConverter) 를 통해 Cursor ai 를 활용하여 PRD 를 바탕으로 신규 기능을 구현하는 과정(6시간)

3번째 실습에서는 요구사항만을 가지고 그에 맞는 API 와 코드를 생성해 나가는 것을 고려했습니다.
요구사항 전체를 입력하는 경우 동작하기는 하나 품질이 높은 코드가 만들어지지 않을 가능성이 높은데요.
세부적인 요구사항들을 하나 하나 변경해 가면서 코드를 완성해 나가는 것을 생각했습니다.
README.md 파일 참고

- 사용자가 입력한 길이(단위:값)를 기반으로, 해당 값을 다른 모든 단위로 변환해 출력하는 프로그램.
- 새로운 단위를 추가할 때 기존 코드의 변경이 최소화되도록 설계한다.
- 각 단위 변환 로직은 테스트 코드로 검증한다.



Unit
Converter

연습 #3 — UnitConverter 요구사항 & OCP/SRP 설계

연습 #3

PRD 요약 (README.md 기반)

	입력	meter:2.5 형식 파싱
	기본 단위	meter · feet · yard
	변환 비율	1m = 3.28084ft = 1.09361yd
	품질	OCP 준수 + SRP 준수 + 입력 검증(음수, 잘못된 형식, 없는 단위)
	추가①	외부 설정파일(JSON/YAML)에서 비율 로드
	추가②	사용자가 동적으로 단위 등록 (1 cubit = 0.4572 meter)
	추가③	출력 포맷 선택: JSON / CSV / 표(table)

💡 Cursor Composer(Ctrl+I): "OCP를 만족하는 인터페이스 구조로 UnitConverter를 구현해줘. 새 단위 추가 시 기존 코드 수정 없이 가능하도록."

🕒 6시간 Activity 구성

0.5h 코드 & 요구사항 분석

2h 기본/품질 요구사항 구현

0.5h 단위 테스트(TC) 작성

2h 추가 요구사항 구현 + TC

1h 회고 & 발표

연습 #3 — UnitConverter 요구사항 & OCP/SRP 설계

연습 #3



OCP/SRP 설계 구조 (C++ & Java)



C++17 - 전략 패턴

```
// 순수 가상 함수로 OCP 구현
class ILengthUnit {
public:
    virtual ~ILengthUnit() = default;
    virtual double toMeter() const = 0;
    virtual std::string name() const = 0;
};
// map<string, unique_ptr<ILengthUnit>>
// 새 단위: 파생 클래스만 추가
```



Java - 인터페이스 설계

```
// OCP: 새 단위 추가 시 코드 수정 없음
interface LengthUnit {
    String getName();
    double toMeter();
}
class Meter implements LengthUnit { ... }
class Feet implements LengthUnit { ... }
class Yard implements LengthUnit { ... }
// SRP: UnitConverter = 변환만 담당
```


연습 #3 — Cursor AI 구현 프롬프트 & 추가 요구사항

연습 #3

🖱 단계별 Cursor AI 프롬프트

기본 구현

@Codebase UnitConverter를 OCP/SRP 원칙으로 구현해줘.
입력: "meter:2.5" → 출력: 모든 단위로 변환 결과
지원 단위: meter, feet, yard
비율: 1m=3.28084ft, 1m=1.09361yd
입력 검증: 음수/잘못된 형식/없는 단위 처리
LengthUnit 인터페이스 + 각 단위 구현 클래스로 분리해줘.

테스트 작성

@UnitConverter.java 단위 변환 기능의 JUnit 5 테스트를 작성해줘:
1. meter→feet 변환 정확도 (소수점 5자리)
2. feet→yard 변환 (meter 기준 역산)
3. 음수 입력 시 IllegalArgumentException
4. 없는 단위 입력 시 예외 처리
5. 경계값: 0, 매우 큰 값

⚙ C++17 동일 구현 포인트

// C++ 추가 요구사항 구현

```
// ① JSON 외부 설정 (nlohmann/json 사용)
#include <nlohmann/json.hpp>
auto units = nlohmann::json::parse(file);
double ratio = units["feet"].get<double>();
```

```
// ② 동적 단위 등록
registry["cubit"] = 0.4572; // meter 기준
```

```
// ③ 출력 포맷 선택
void printResult(const Results& r, Format fmt) {
    if (fmt == Format::JSON)    printJSON(r);
    if (fmt == Format::CSV)    printCSV(r);
    if (fmt == Format::TABLE)  printTable(r);
}
// CMakeLists.txt에 nlohmann_json 추가 필요
```

연습 #3 — Cursor AI 구현 프롬프트 & 추가 요구사항

연습 #3

추가 요구사항

@Codebase 다음 3가지 추가 기능을 구현해줘:

- ① JSON/YAML 외부 설정에서 변환 비율 로드
(units.json: {"meter":1.0,"feet":3.28084,...})
 - ② 동적 단위 등록: "1 cubit = 0.4572 meter" 입력
 - ③ 출력 포맷 선택: --format json|csv|table 옵션
- 각 기능에 대한 테스트 코드도 함께 작성해줘.

출력 예시 (표 포맷)

실행 예시

```
$ java -jar converter.jar "meter:2.5" --format table
```

단위	입력값	변환값
meter	2.5	2.5
feet	2.5	8.20
yard	2.5	2.73

✅ 추가 요구사항 구현 후 반드시 TC 작성: JSON 로드 실패 케이스 / 중복 단위 등록 / 포맷별 출력 정확도 검증.

연습 #4

Feedback Analyzer Legacy 개선

레거시 이해 → 테스트 추가 → 버그 수정 → 신규 기능

⚙ C++17 (주)

🐱 Java 17 (참고)

4

연습 #4 — Feedback Analyzer 개요 & 레거시 문제점

연습 #4

C++ Feedback Analyzer — 파일별 역할 이해

main.cpp

HTTP 서버 진입점
httplib 서버·라우팅·HTML 렌더링

TextAnalyzer.h/.cpp

감성 분석 (sent)
키워드 분석 (kw)
전역 결과 캐싱

Filters.h/.cpp

감정/키워드 필터링
정적 키워드 맵 (S_KEYWORDS)
필터 로직 (fil)

Session.h/.cpp

전역 피드백 상태 관리
currentFeedbacks 저장
세션 초기화 stub

Constants.h/.cpp

감성·카테고리 키워드
정적 맵 초기화 (init)
한국어 키워드 사전

Logger / UI / FileHandler

Logger: 타임스탬프 로그
UIComponents: 카테고리 목록
FileHandler: 결과 저장 stub

연습 #4 — Feedback Analyzer 개요 & 레거시 문제점

연습 #4

Java Feedback Analyzer 코드 구조 (12개 파일) — 문제점 내장된 레거시

파일명	역할	문제점	심각도
DemoApplication	Spring Boot 앱 시작점 (mvn spring-boot:run)	정상	OK
FeedbackController	Controller + 데이터 로직 혼합	SRP 위반	Critical
TextAnalyzer	감정·키워드 분석 (중복 많음)	중복·Long Method	Critical
FileHandler	CSV 읽기/저장 + UI 로직 혼합	SRP 위반	Critical
Constants	감정·분류 키워드	하드코딩	Warning
Filters	필터링 로직 (데이터 흐름 불명확)	버그: '중립' 필터 오류	Critical
Logger	로그 출력 (print 남용)	과도한 print 출력	Warning
Session	전역 변수 상태 관리	전역 변수 남용	Warning
DemoApplicationTests	테스트 흉내만 낸 파일	실제 테스트 없음	Critical
index.html	Thymeleaf UI	UI-로직 혼재	Warning



시작 전: Cursor Chat → @Codebase "이 Spring Boot 프로젝트의 전체 구조와 문제점을 분석해줘" → analysis.md 저장 후 단계별 진행.

연습 #4 — Feedback Analyzer 개요 & 레거시 문제점

연습 #4

실제 코드에서 발견되는 6가지 개선 포인트

① 축약된 메서드명

sent(), kw(), fil(), fil_data
→ analyzeSentiment(), analyzeKeywords()로 개선

TextAnalyzer / Filters

② 중복 키워드 등록

Constants.cpp에 동일 키워드가
두 번씩 삽입 (긍정·부정 모두)

Constants.cpp

③ 비공개 비즈니스 로직

containsAny() 가 TextAnalyzer와
Filters 양쪽에 중복 정의

TextAnalyzer.h / Filters.h

④ 전역 상태 남용

fil_data 전역변수, globalSent/
globalKw static 멤버 — 테스트 불가

main.cpp / TextAnalyzer

⑤ 의미 없는 Session

getOldDataFromSession()이 key를
무시하고 항상 같은 데이터 반환

Session.h

⑥ FileHandler stub

saveResult()가 cout만 출력,
실제 파일 저장 로직 없음

FileHandler.h

연습 #4 — 8단계 학습 가이드 & Cursor AI 프롬프트

연습 #4

1

프로젝트 개요 & 준비

1h

@Codebase 전체 구조·미션 안내 분석해줘

3

버그 수정 3건

1.5h

@Filters.java '중립' 필터 버그를 수정해줘.
@index.html 텍스트 입력창을 multiline으로 수정.
@Logger.java 로그 레벨(warning/error)을 UI에서 제어하도록 수정.

5

긴 함수 & 중복 제거

1.5h

@TextAnalyzer.java 20줄 이상 메서드를 추출하고, 중복 로직을 공통 메서드로 통합해줘. SRP에 따라 클래스 분리 제안도 해줘.

7

추가 기능 구현

3h

① test_feedback_trend.csv로 Trend 시각화 추가
② 감정분석 필터를 File DB로 관리하도록 변경

2

테스트 구조 개선 (90%+)

2h

@DemoApplicationTests.java 커버리지 90% 달성을 위한 JUnit 5 테스트를 작성해줘. TextAnalyzer.Filters.FileHandler 각 클래스별 TC 포함.

4

네이밍 & 매직넘버 개선

1h

@Constants.java @TextAnalyzer.java 매직 넘버를 상수로 추출하고, 의미없는 변수명을 도메인 용어로 개선해줘.

6

SRP 리팩토링

1h

@FeedbackController.java 데이터 로직을 Service 클래스로 분리해줘. Controller는 HTTP 처리만 담당하도록.

8

팀 리뷰 & 발표

2h

@Codebase 리팩토링 전후 비교 보고서를 Markdown으로 작성해줘. Before/After 코드 포함.



단계 2~6은 각 완료 후 git commit -m "refactor/test/fix: [내용]" → push → PR 생성. 모든 PR은 merge 하지 말고 Review만 수행.

연습 #4 — 테스트 추가 — Google Test (C++)

연습 #4

⚙ C++17 — Google Test

```
#include <gtest/gtest.h>
#include "TextAnalyzer.h"
#include "Constants.h"

class TextAnalyzerTest : public ::testing::Test {
protected:
    void SetUp() override {
        Constants::init();
    }
};

TEST_F(TextAnalyzerTest, PositiveSentiment) {
    std::vector<Feedback> feedbacks = {
        Feedback("배송이 매우 빠르고 좋았습니다")
    };
    TextAnalyzer analyzer;
    auto result = analyzer.analyzeSentiment(feedbacks);
    EXPECT_EQ(result["긍정"], 1);

    EXPECT_EQ(result["부정"], 0);
}
```

핵심 패턴

Fixture 사용

TEST_F + SetUp()으로
Constants::init() 공통 초기화

단일 책임 검증

긍정·부정·중립 각각
독립 TEST_F 케이스 작성

경계값 테스트

빈 feedbacks, 혼합 감성
키워드 없는 텍스트 검증

CMake 연동

find_package(GTest)
target_link_libraries 설정

연습 #4 — 테스트 비교 — JUnit 5 (Java 참고)

연습 #4

동일한 로직을 Java로 작성하면? — 구조적 차이 이해

C++ — Google Test

```
// SetUp: Constants::init() 호출
TEST_F(AnalyzerTest, FilterPositive) {
    Filters filters;
    Filters::
    initFilterKeywords();
    auto res = filters.filter(
        feedbacks, "긍정", "전체");
    ASSERT_EQ(res.size(), 1u);
}
```

Java — JUnit 5 + Mockito

```
@ExtendWith(MockitoExtension.class)
class FilterServiceTest {
    @InjectMocks FilterService service;

    @Test
    @DisplayName
    ("긍정 필터 정상 동작")
    void
    filterPositive() {
        var res = service
            .filter(feedbacks, "긍정");
        assertThat(res).hasSize(1);
    }
}
```

비교 항목	C++ (Google Test)	Java (JUnit 5)
테스트 픽스처	TEST_F + SetUp()	@BeforeEach
단언문	EXPECT_EQ / ASSERT_EQ	assertThat (AssertJ)
Mocking	없음 (수동 stub)	@Mock / @InjectMocks
빌드 연동	CMake FetchContent	Maven / Gradle
실행 속도	매우 빠름 (네이티브)	JVM 기동 시간 포함

연습 #4 — 버그 수정 — 중복 코드 제거

연습 #4

BUG ① — Constants.cpp 중복 키워드

✗ Before (버그)

```
SENTIMENT_KEYWORDS["긍정"] = {  
    "좋아요", "만족", "감사", "최고",  
    // ... 13개  
    "좋아요", "만족", "감사", "최고",  
    // ← 동일 항목 반복! (중복)  
};
```

✓ After (수정)

```
SENTIMENT_KEYWORDS["긍정"] = {  
    "좋아요", "만족", "감사", "최고",  
    "좋은", "훌륭", "추천", "좋았어요",  
    "좋습니다", "최고입니다",  
    "감사합니다", "만족스럽"  
}; // 중복 제거
```

BUG ② — containsAny() 중복 정의 → 공통 유틸 추출

```
// ✓ TextUtils.h — 공통 헬퍼로 추출  
namespace TextUtils {  
    inline bool containsAny(const std::string& text,  
                           const std::vector<std::string>& kws) {  
        return std::any_of(kws.begin(), kws.end(),  
                           [&text](const auto& kw){ return text.find(kw) != std::string::npos; });  
    }  
}  
// TextAnalyzer.h, Filters.h 양쪽에서 #include "TextUtils.h" 후 재사용
```

연습 #4 — 리팩토링 — 명명 & 설계 개선

연습 #4

① 메서드명 명확화 (TextAnalyzer)

기존 (레거시)	개선 후
sent()	analyzeSentiment()
kw()	analyzeKeywords()
fil()	filter()
fil_data (전역)	FilterResult (반환값)
getOldDataFromSession()	getCurrentFeedbacks()

Java 비교

Java도 동일 원칙 — calculateSentiment(), applyFilter() 등
명사/동사 혼용 없이 동사+명사 구조 유지 (Java Convention)

② Session 설계 개선

```
// ❌ Before: key 무시, 이름 혼란
static vector<Feedback>& getOldDataFromSession(const string&
key) {
    return currentFeedbacks; // key 미사용
}

// ✅ After: 명확한 인터페이스
class FeedbackSession {
public:
    static vector<Feedback>&
    getCurrent();
    static void
    update(const vector<Feedback>&);
    static void
    clear();
};
```

✅ 개선 효과: 테스트 가능 (testable), 의도가 명확한 API 제공
전역 변수 의존 제거 → 단위 테스트에서 clear() 후 독립 실행 가능

연습 #4 — 신규 기능 — FileHandler 실제 구현

연습 #4

cout stub → 실제 CSV 파일 저장 + 에러 처리

❌ Before — 콘솔 출력만 (stub)

```
void saveResult(const vector<Feedback>& data) {  
    cout << "saveResult" << data.size();  
    for (const auto& f : data)  
        cout << f.getText();  
}
```

✅ After — 실제 CSV 저장

```
bool saveToCsv(const vector<Feedback>& data,  
               const string& path) {  
    ofstream ofs(path);  
    if (!ofs.is_open())  
        return false;  
    ofs << "\xEF\xBB\xBF" << "text\n"; // UTF-8 BOM  
    for (const auto& f : data)  
        ofs << f.getText() << "\n";  
    return true;  
}
```

🐼 Java 비교 — Files.write() (NIO2)

```
// Java — Files.write + Stream API  
public boolean saveToCsv(List<Feedback> data, Path path) throws IOException {  
    var lines = Stream.concat(  
        Stream.of("text"),  
        data.stream().map(Feedback::getText)  
    ).collect(Collectors.toList());  
    Files.write(path, lines, StandardCharsets.UTF_8);  
    return true;  
}
```

연습 #4 — 신규 기능 — 감성 분석 정확도 개선

연습 #4

단순 키워드 매칭 → 가중치 기반 스코어링으로 업그레이드

⚙ C++17 — 가중치 스코어링

```
struct SentimentScore { int positive{0}, negative{0}; };

string classifyWeighted(const string& text) {
    SentimentScore score;
    for (const auto& kw : SENTIMENT_KEYWORDS["긍정"])
        if (text.find(kw) != npos) score.positive++;
    for (const auto& kw : SENTIMENT_KEYWORDS["부정"])
        if (text.find(kw) != npos) score.negative++;

    if (score.positive > score.negative) return "긍정";
    if (score.negative > score.positive) return "부정";
    return "중립";
}
```

기존 방식 문제

첫 번째 매칭 키워드로만 결정
긍정+부정 혼합 텍스트 오분류
중립 판단이 마지막 else fallback

개선 효과

모든 키워드 카운트 후 비교
혼합 감성 텍스트 정확도 향상
TEST_F로 각 케이스 독립 검증

1

레거시 이해

- ▶ 파일 구조·의존성 파악
- ▶ 문제점 6가지 진단
- ▶ 테스트 없는 코드 위험

2

테스트 추가

- ▶ Google Test 픽스처
- ▶ 경계값·혼합 케이스
- ▶ CMake 빌드 연동

3

버그 수정

- ▶ 중복 키워드 제거
- ▶ containsAny 공통화
- ▶ Session API 개선

4

신규 기능

- ▶ FileHandler CSV 저장
- ▶ 가중치 감성 분석
- ▶ 명명 규칙 일관화

종합 — Markdown 보고서 작성 & Cursor AI 활용 회고

Cursor AI로 Markdown 보고서 자동 생성

보고서 생성 프롬프트 (Chat)

Cursor.AI 분석 결과를 바탕으로 보고서를 작성해줘.
출력 형식: Markdown (.md)

구조:

```
# 코드 리뷰 및 개선 보고서
## 1. 분석 배경 (프로젝트·목적)
## 2. Cursor AI 분석 요약 (항목별 표)
## 3. 개선/처리 내역
| 피드백사항 | 수정 조치 | 결과 |
## 4. 전후 코드 비교 (java/cpp 코드 블록)
## 5. 효과 및 향후 과제
```

"이 결과를 report.md 파일에 저장해줘"
순수 Markdown만 출력하고 AI 설명은 제외해줘.

회고 & 발표 가이드 (공통)

실습 목표 달성도

계획 대비 실제 구현 범위, TC 커버리지 달성률

코드 품질 Before/After

코드 스멜 수 감소, 테스트 커버리지 변화, 복잡도 지표

AI 활용 경험

도움이 된 순간: 대규모 리팩토링, 패턴 적용
한계: 도메인 지식 부족, 컨텍스트 손실

TC 추가 효과

버그 발견 케이스, 회귀 방지 경험, 리팩토링 자신감

클린코드·리팩토링

가장 임팩트 있었던 변경, 어려웠던 점, 팀 공유 팁

종합 — Markdown 보고서 작성 & Cursor AI 활용 회고



연습별 필수 제출 .md 파일

#1 TDD_TV

analysis.md · review_report.md · pr_checklist.md

#2 SHealth

code_smell.md · refactoring.md · unittest.md · feature.md · retrospective.md

#3 UnitConverter

prd.md · implementation.md · test_report.md

#4 FeedbackAnalyzer

analysis.md · bug_fix.md · refactoring.md · feature.md · report.md



회고 Cursor 프롬프트: "이번 실습에서 Before/After 코드 비교와 AI 활용 경험을 5분 발표 자료용 Markdown으로 정리해줘."

AI가 빠르게 짜준 코드보다 내가 이해한 코드가 더 오래 살아남는다.

- ✓ 연습 #1: 기존 코드를 Cursor AI로 체계적으로 리뷰하는 10영역 접근법
- ✓ 연습 #2: 코드 스멜 탐지 → 리팩토링 → TDD → 기능 개선의 완전한 사이클
- ✓ 연습 #3: PRD만으로 OCP/SRP를 만족하는 확장 가능한 구조를 AI와 설계
- ✓ 연습 #4: 레거시 코드를 빠르게 이해하고 안전하게 개선하는 전략

수 고 하 셧 습 니 다 🎉